Comparative Performance Analysis of Parallel Implementations for Accelerated Streamline Visualization in Vector Field Analysis

Nate Stott, Rylei Mindrum, Ethan Gee

***Abstract*—Streamline visualization is a fundamental tool for analyzing complex vector fields, yet it often faces significant computational overhead due to the intensive numerical integration and irregular data-access patterns required for tracing paths across high-resolution grids. This study presents a comprehensive suite of implementations aimed at accelerating streamline visualization through various parallel computing paradigms. We developed and evaluated a vector field analyzer across several architectures, namely: sequential CPU, shared-memory parallelism using Pthreads and OpenMP, distributed-memory systems with OpenMPI, and massively parallel GPU acceleration through CUDA, as well as a hybrid approach utilizing distributed CUDA with OpenMPI. Our analysis focuses on the scalability and speedup of each implementation, comparing performance across various example datasets at a variety of field sizes and streamline complexities. Results demonstrate significant performance gains over sequential implementations, with the distributed GPU implementation showing the best performance increase.**

***Index Terms*—Streamline Visualization, Vector Field Analysis, Parallel Computing, OpenMP, OpenMPI, CUDA, GPU Acceleration, Pthreads.**

I. Introduction

Vector field analysis is a cornerstone of scientific visualization, providing critical insights into systems that involve particle interactions such as aerodynamics and fluid dynamic modeling, and areas that exhibit consistent forces such as gravitational modeling and magnetic fields.

These fields consist of a grid of discretely spaced points that are each assigned a vector. This vector serves to store the strength and direction of motion around that point at a given instance.

To be able to effectively understand and analyze these fields, researchers rely on various models of these fields to identify key attributes such as vorticity, divergence, critical points, and flux.

A. Streamline Analysis

A streamline represents the path that an object would take if it moved through a static vector field. The path is traced by starting from a given vector and always proceeding in the direction of the current vector iteratively. By connecting these directions throughout the field, streamlines illustrate how a substance or particle would flow at a particular instant. This approach provides valuable insight into the topology of a given vector field. This allows researchers to easily identify the overarching flow of a field, its sources and sinks, and other points of criticality.

Formally, a streamline is a curve that follows the direction of the vector field at every point, representing the path an object would take if it moved through the field at a given instant. For a steady vector field $\mathbf{v}(\mathbf{x})$, a streamline $\mathbf{x}(s)$ is defined by the differential equation:

$$\frac{d\mathbf{x}}{ds} = \mathbf{v}(\mathbf{x}(s)) \tag{1}$$

where s is a parameter along the curve.

These differ from pathlines, which seek to track a particle's progress over time. In comparison, streamlines allow for a more comprehensive snapshot of entire vector fields at a static level; they can also be used to show how a given field changes over time, which is highly relevant to disciplines like magnetism. This broader reach leads to a far higher complexity for streamline tracing in comparison to pathlines.

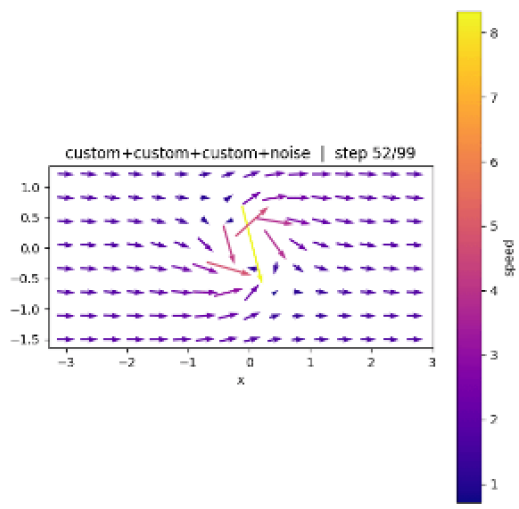

Fig. 1. A discrete vector field grid representing force and direction at each point.

B. Computational Challenges

Despite their utility, streamline visualization faces significant computational overhead.

When analyzing large-scale datasets, such as those generated by high-fidelity simulations or astrophysical models, the computational cost of seeding and tracing thousands of streamlines becomes a bottleneck.

6030 High Performance Computing, Utah State University.

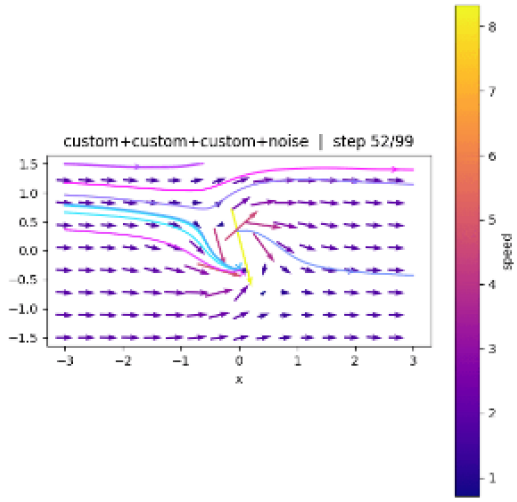

Fig. 2. A vector field grid with high-velocity streamlines highlighted to illustrate flow patterns.

A major challenge arises from the irregular memory access patterns inherent in following a path through a grid. Modern systems utilize caching to help ease the computational slowdown inherent to high I/O operations. As streamlines are traced, vector points frequently point to subsequent vectors that fall outside of contiguous memory. This leads to inefficient use of the cache and increased memory latency, which can significantly degrade performance on modern cache-hierarchical architectures.

The joining process of streamlines themselves also introduces challenges as it requires a store of information for each streamline that must be communicated globally to ensure that streamlines are treated as a continuous series.

Additionally, parallel implementations rely heavily on partitioning the dataset into smaller subdomains that can be processed independently. Due to the inherent out-of-bounds access of vectors, parallel processes are forced to communicate across processes. This introduces significant communication overhead. This is especially apparent in GPU-accelerated and distributed implementations where data must be communicated and synchronized between processing units that don't have direct access to the isolated memory of other processes.

Together, these factors make efficient and scalable streamline computation a challenging problem in high-performance vector field analysis.

C. Contributions

This paper presents a research tool developed in C++17 for the accelerated analysis of 2D vector fields.

We address the computational bottleneck of streamline tracing by implementing and evaluating several parallel computing strategies.

Our primary contributions include:

- A modular simulator for generating procedural vector fields (vortices, noise, and custom expressions) stored in HDF5 format.
- A suite of parallel analyzer implementations, ranging from shared-memory models (Pthreads, OpenMP) to distributed-memory (OpenMPI) and massively parallel GPU acceleration (CUDA).
- A comparative performance study demonstrating the scalability and speedup of these implementations across various field resolutions and streamline densities.

II. Methodology

A. Core Components

1) Vector Representation: At the most fundamental level, the system utilizes a `Vec2` class to represent 2D vectors. This class encapsulates two floating-point components, x and y, and essential mathematical operations:

- **Vector Arithmetic:** Standard operations for vector addition and scalar multiplication.
- **Normalization:** Methods to calculate the magnitude and derive unit vectors to allow for easy directional comparison.

2) Field Representation: Vector fields are represented as discretely spaced vectors stored within a grid structure.

The primary data structures are:

- `FieldSlice`: A 2D grid of discretely spaced vectors.
- `FieldTimeSeries`: A collection of field slices associated with physical-space bounds $(x_{\min}, x_{\max}, y_{\min}, y_{\max})$.
- `FieldGrid`: Serves as the high-level manager for a single time-step's vector field, handling all the necessary calculations for streamline computations.

The `FieldReader` component handles the deserialization of these fields from HDF5 files using the HighFive library. It extracts `vx` and `vy` datasets into the internal grid representation, ensuring that physical dimensions and time-series data are correctly mapped to the discrete grid indices.

3) Grid Management and Streamline Tracing: Each `Streamline` object stores an ordered list of grid coordinates (`row, col`) that constitutes a path. This decouples the mathematical vector data from the state of the streamline tracing algorithm by maintaining a corresponding grid of `Streamline` pointers. This abstraction allows the tracing logic to handle merging and path extension while keeping the underlying vector field read-only, which is critical for thread safety in parallel implementations.

B. Streamline Tracing Algorithm

The core logic for analyzing vector fields involves a discretized streamline tracing algorithm that maps continuous vector directions onto the underlying computational grid. To manage the connectivity and merging of these paths efficiently, the system employs a Disjoint Set Union (DSU) or Union-Find algorithm.

1) Discretized Subsequent Vector Calculation: For each cell (i, j), the algorithm computes the target subsequent vector by first determining the physical spacing between grid cells based on the field boundaries and grid dimensions. It then advances the cell's physical position by the vector's (x, y) components and converting back to grid indices using its knowledge of grid spacing.

2) Connectivity Discovery via Union-Find to allow for Path Compression: Using grid indices allows for the vector field grid to operate as a collection of disjoint sets where each disjoint set is a streamline. The algorithm treats each grid cell as an element in a disjoint-set structure. Connectivity is established by performing a `unite` operation between each cell and its calculated downstream subsequent vector.

The `unite` operation serves as the primary mechanism for establishing connectivity between grid cells. In parallel contexts, this is performed using atomic synchronization primitives (e.g. barrier) to ensure that multiple threads can safely update the shared set structure without requiring heavy-weight locks. To maintain high performance as the grid size increases, the algorithm employs path halving during the root discovery process, which incrementally flattens the tree structure and ensures that subsequent connectivity checks remain efficient. This happens iteratively until it's subsection remains stable across passes.

This approach offers several advantages:

- **Efficient Merging:** Streamlines that converge into the same cell are naturally merged into the same set, representing a single flow component.
- **Path Compression:** The `findRoot` operation, combined with path compression or halving, ensures that determining the component ownership of any cell is nearly constant time ($O(\alpha(n))$).

After all cells have been processed, the resulting disjoint sets represent the unique streamlines within the field. Boundary effects and terminal points are handled naturally: if a vector points back to its own cell (after clamping), it remains its own root.

C. Serial CPU Implementation

The baseline implementation, `sequential CPU`, serves as a reference for both correctness and performance metrics. It is implemented using modern C++17 standards and processes the vector field grid in a single pass using nested loops to iterate through rows and columns to identify it's subsequent vector. Then each cell invokes the `traceStreamlineStep` method, which takes the original and subsequent vector and adds the subsequent vector to the streamline of the original.

D. Shared-Memory Parallelism CPU Implementation

To leverage multi-core architectures, we developed parallel implementations using POSIX Threads and OpenMP. A significant challenge in parallelizing the streamline tracing algorithm is the stateful nature of the Union-Find structure; multiple threads performing `unite` operations could result in race conditions. To address this, both shared-memory implementations employ a multi-pass strategy:

1) **Parallel Subsequent Vector Precomputation:** The algorithm first calculates the destination subsequent vector for every cell in the grid. Since this step only reads from the vector field and writes to an intermediate buffer, it is perfectly parallelizable.
2) **Parallel Root Discovery:** After the sequential pass, the `findRoot` operation is performed in parallel for all cells to determine their final component assignments.

1) POSIX Threads (Pthreads): The grid is partitioned into chunks where each thread is assigned a contiguous range of rows. This minimizes the overhead of thread management while ensuring that each thread operates on a disjoint memory region in the subsequent vector buffer. The implementation explicitly manages thread creation and joining, handling potential errors and ensuring that all threads terminate before proceeding to the sequential pass.

2) OpenMP: This works identically to the POSIX threads implementations with one modification. We employ `#pragma omp parallel for` directive with a `collapse(2)` clause to allow for easy segmentation and load balancing across our nested loops. T

E. Massively Parallel GPU Acceleration with CUDA

The CUDA implementation offloads the computationally intensive tasks of downstream successor computation and component discovery to the GPU, leveraging its high throughput to process many grid cells simultaneously. Each cell is handled by an independent CUDA thread, allowing the field to be analyzed in parallel at a much finer granularity than in the CPU implementations. While it shares the multi-pass structure of the parallel CPU implementations, it extends the parallelism to connectivity analysis through the use of device-side synchronization primitives.

1) **Parallel Successor Computation:** A CUDA kernel assigns one thread to each grid cell to compute that cell's downstream successor in parallel.
2) **Parallel Union-Find:** Unlike the shared-memory CPU versions, which rely on a sequential merging pass, the CUDA implementation resolves streamline connectivity on the device using a parallel Union-Find algorithm. Device-side synchronization is handled with `atomicCAS` (Compare-And-Swap), which ensures thread-safe updates during set merging.
3) **Path Compression and Labeling:** A compression pass updates each cell so that it points directly to its representative root. This reduces lookup cost, improves consistency of the disjoint-set structure, and yields a compact component-based representation of the streamline topology.
4) **Path Reconstruction for Analyzer Output:** Because the analyzer ultimately expects explicit streamline paths rather than only component labels, the GPU-generated successor structure is reconstructed into host-side streamline paths after the device computation completes. This allows the GPU-native implementation to produce the same form of output as the other solver backends while preserving the benefits of GPU-side connectivity analysis.

F. Distributed-Memory CPU Implementation

The distributed-memory implementation using MPI introduces a significant challenge not present in the shared-memory implementations: processes do not share an address space, so the sequential Union-Find pass cannot operate globally across all ranks. To address this, each rank is assigned a contiguous range of rows and independently computes and

merges the downstream subsequent vectors for its assigned cells. This correctly resolves all connections whose source cell falls within the rank's row range, but connections that cross rank boundaries are only partially recorded — the source rank knows about the connection, but the destination rank does not. We resolve these cross-boundary connections through an iterative `MPI_Allreduce` loop, where each round broadcasts the current root of every cell across all ranks and takes the global minimum. Any rank that observes a globally lower root for a cell than its own merges those two components locally via the `unite` operation, propagating that connectivity inward. Each round resolves connections one rank boundary further, so at most p rounds are needed for a field partitioned across p ranks, with an early-exit condition terminating the loop as soon as no rank reports any changes. Once all ranks have converged to a consistent global state, path reconstruction proceeds identically to the shared-memory implementations, with each rank building paths for its assigned cell range and the results gathered to rank 0.

G. Distributed-Memory GPU Implementation (Hybrid)

This hybrid approach leverages the high throughput of GPU acceleration while maintaining the horizontal scalar ability ability of distributed-memory clusters, effectively shifting the computational bottleneck from local tracing to global network communication. This approach essentially offloads the local computation of each MPI rank to a dedicated GPU, merging the multi-pass architecture of our CUDA implementation with the domain-decomposition strategy of our MPI solver.

By assigning each rank to a specific local GPU and a corresponding sub-grid of the vector field, the system achieves significant acceleration in the initial subsequent vector calculation phase. Global connectivity is then resolved using the same iterative reconciliation strategy employed in the distributed CPU version, ensuring that streamlines crossing node boundaries are correctly identified and merged. This hybrid approach leverages the high throughput of GPU acceleration while maintaining the horizontal scalability of distributed-memory clusters, effectively shifting the computational bottleneck from local tracing to global network communication.

H. Experimental Design

This section outlines the setup and procedures for the scaling studies and performance evaluations conducted across different architectures.

III. Results

This section presents the findings of our comparative performance analysis across the various parallel implementations.

A. Experimental Setup

All performance evaluations were conducted on the lonepeak cluster at the Center for High Performance Computing (CHPC) using four compute nodes.

The benchmarks involved scaling across mutliple gird sizes size, as well as a primary analysis of a primary 1024×1024 *vortex* field.

Each test was measured against a single static state of these fields. Timing was measured inside of main.cpp to avoid measuring the loadtimes of the various libraries. To ensure statistical significance, the analysis for each implementation was repeated five times and the resulting metrics were averaged. Furthermore, the correctness of each parallel implementation was validated by comparing its output against the sequential reference implementation. In all cases, the results achieved bitwise parity with the serial output, confirming identical results across all implementations.

B. Shared-Memory Threading Comparison

This study compares the performance of POSIX threads (Pthreads) against OpenMP across varying thread counts to evaluate threading overhead and efficiency.

TABLE I
Threading Performance: Pthreads vs. OpenMP

# Threads	POSIX Threads Runtime (ms)	OpenMP Runtime (ms)
1 (Serial)	1518.86 (0.00%)	1518.86 (0.00%)
2	1427.36 (6.41%)	1424.25 (6.64%)
4	1479.87 (2.63%)	1876.70 (-19.07%)
8	1916.70 (-20.76%)	1862.73 (-18.46%)

The results indicate that both Pthreads and OpenMP implementations provide a slight performance improvement at low thread counts, but performance improvement degrades as the number of threads increases as the overhead of the parralel disjoin union join grows.. OpenMP achieved its best runtime at 2 threads, whereas adding more threads significantly increased the execution time for both implementations. This inverse scaling suggests that the overhead of thread management and the sequential bottleneck of the merging pass dominate the computation as parallelism increases.

Applying Amdahl's Law to the OpenMP implementation at 2 threads, we observe a speedup of $S \approx 1.07$. Using the relation $S(n) = \frac{1}{(1-p)+p/n}$ for $n = 2$, we estimate a parallel fraction $p \approx 0.13$. This implies that only 13% of the total execution time is effectively parallelized. For $n > 2$, the speedup $S < 1$ indicates that the system has entered a regime where synchronization overhead and resource contention outweigh the parallel gains, a common phenomenon when the serial fraction is high and the workload per thread is insufficient to hide thread creation latency.

C. GPU Block Size Analysis

TABLE II
GPU Performance vs. Block Size

Block Size	Runtime (ms)	Speedup
128×128	963.29	1.58
256×256	800.86	1.90
512×512	800.17	1.90

The GPU performance analysis demonstrates that increasing the block size from 128 to 256 threads provides a substantial performance boost, reducing runtime from 963.3 ms to 800.9 ms. This improvement suggests that a block size of 256 better

utilizes the GPU's streaming multiprocessors and improves latency hiding. Moving to a block size of 512 threads yielded a negligible improvement, reaching 800.2 ms. Indicating that the system has reached a point of saturation where additional threads per block do not provide further benefits.

Analysis of the GPU performance reveals a parallel efficiency significantly higher than the CPU implementations. With a maximum speedup of 1.90×, Amdahl's Law (assuming $n \to \infty$ for the thousands of CUDA threads) yields $S \approx \frac{1}{1-p}$, implying a parallel fraction $p \approx 0.474$. This indicates that approximately 47.4% of the total workload is successfully accelerated by the GPU. The remaining 52.6% is comprised of sequential overhead, including host-to-device memory copies and the initial field serialization, which limit the theoretical maximum speedup regardless of GPU thread count.

D. Distributed CPU vs. Distributed GPU Scaling Study

TABLE III
DISTRIBUTED SCALING COMPARISON

Nodes	Distributed CPU Runtime (ms)	Distributed GPU Runtime (m
2 Nodes	1992.49	3258.99
4 Nodes	3962.65	4516.31

TABLE IV
OVERALL PERFORMANCE COMPARISON

Technology	Runtime (ms)	Speedup
Serial	1518.86	1.00
Pthreads	1427.36	1.06
OpenMP	1424.25	1.07
OpenMPI	1992.49	0.76
CUDA	800.17	1.90
Hybrid (MPI+CUDA)	3258.99	0.47

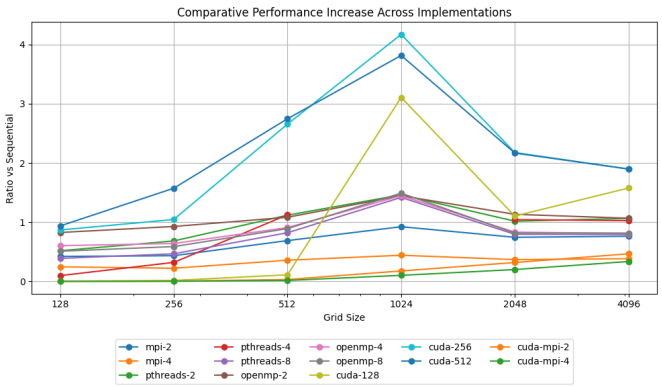

Fig. 3. Runtime performance comparison for various parallel implementations.

The distributed-memory implementation underperformed in comparison to its shared memory counterparts. While the neighbor calculation phase was able to be scaled effectively due to the isolated calculations, the parallel path merging phase relies heavily on accessing data from other parts of the vector field when working across the boundaries. This high overhead stems from the requirement to reconcile streamline connectivity across process boundaries. Because each node only possesses local knowledge of its assigned sub-grid, resolving global paths requires iterative synchronization to converge the distributed Disjoint Set Union (DSU) structure. As the node count increases, the latency associated with these global reductions and data transfers becomes a dominant factor, limiting the overall scalability of the distributed CPU implementation.

Speedup analysis for the distributed implementations shows a significant performance drop as the node count increases. The 2-node CPU implementation achieved a speedup of $S \approx 0.76$, which further degraded to $S \approx 0.38$ at 4 nodes. Similarly, the hybrid (MPI+CUDA) implementation achieved $S \approx 0.47$ at 2 nodes and $S \approx 0.34$ at 4 nodes. A speedup less than unity confirms that the communication overhead $O(n)$ grows faster than the parallel speedup for this algorithm. In the context of Amdahl's Law, the communication costs act as a "negative" parallel component that offsets the gains from distributed computation, resulting in a total execution time that exceeds the sequential baseline.

E. Performance Comparison Overview

Table IV provides a high-level comparison of the runtime and speedup for each implementation type.

While shared-memory implementations like Pthreads and OpenMP provide incremental performance gains through task-level parallelism, they are ultimately limited by the sequential nature of single-node CPU execution for this specific workload. While the overall calculations of the neighbor calculation and the union joining of the disjoint sets were able to speed up their respective stages, the sequential portions continue to dominate the runtime. The merging of the disjoint sets into a singular ordered list is inherently sequential and cannot be avoided.

As observed in the scaling study, the requirement for global synchronization of the Disjoint Set Union (DSU) structure across process boundaries introduces a communication latency that is a bottleneck that limits the effectiveness of the parallel DSU at larger node sizes. The overhead of iterative synchronization stages required to reconcile streamline connectivity between sub-grids makes distributed scaling exceptionally difficult.

Furthermore, the irregular data access continues to be a challenge that leads to a high amount of disk I/O that cannot be effectively optimized through caching. This is further complicated by distributed-memory implementations where the communication overhead grows exponentially with each additional node.

IV. RESPONSIBILITIES

A. Nate Stott

- Developed the comprehensive testing and automation pipeline.
- Implemented the procedural vector field simulator with HDF5 support.
- Integrated OpenMPI and developed the necessary communication wrappers to handle distributed data structures and global synchronization.
- Co-authored the CPU-based Disjoint Set Union implementations and optimized for openMPI.

- Developed the Python-based visualization tools for qualitative streamline analysis.
- Tested the implementations on CHPC.

B. Rylei Mindrum

- Architected and implemented both standalone CUDA and hybrid MPI+CUDA streamline solvers.
- Designed the parallel Disjoint Set Union (DSU) implementation for the project. as well as implementing in the CUDA implementation to ensure feasibility utilizing atomic synchronization primitives for thread-safe connectivity discovery.
- Focused on architectural compliance to ensure code reuse across the C++, CUDA, and OpenMPI implementations.
- Tested the implementations on CHPC.

C. Ethan Gee

- Developed the project's foundational data structures, including the `Vec2` representation and the `FieldGrid` management system.
- Implemented the shared-memory parallel solvers using POSIX Threads and OpenMP.
- Co-authored the CPU-based Disjoint Set Union implementations and optimized them for shared-memory environments.
- Authored the final research report and conducted the comprehensive performance and scaling analysis, including the application of Amdahl's Law to the experimental findings.

V. Conclusion

Our comparative analysis reveals that while shared-memory implementations (POSIX Threads and OpenMP) provide incremental performance gains of approximately 6–7% at low thread counts, they are ultimately constrained by a high serial fraction ($\approx 87\%$). The overhead of thread management and the sequential requirements of the final path reconstruction phase limit the scalability of these CPU-based parallel strategies.

In contrast, the massively parallel CUDA implementation achieved the most significant performance improvement, providing a $1.90\times$ speedup over the sequential reference. The GPU's ability to handle millions of cells simultaneously through optimized block sizes and device-side atomic synchronization successfully mitigated much of the tracing overhead, although performance remained bound by host-to-device memory transfer latency and the non-parallelizable serialization of output data.

The distributed-memory implementation using OpenMPI highlighted the significant challenges of communication of memory across isolated processes. The high communication cost associated with iterative synchronization resulted in a speedup of slower than a serial implementation ($0.76\times$).

Crucially, all parallel implementations achieved bitwise parity with the serial reference, demonstrating that performance acceleration can be achieved without compromising the numerical integrity of the flow analysis. Ultimately, while GPU acceleration provides the most immediate path to performance gains, the development of truly scalable distributed streamline visualization requires minimizing global synchronization overhead.

Acknowledgments

We would like to thank the Center for High Performance Computing (CHPC) at the University of Utah for providing the computational resources and technical support that made this research possible.